

hetmacro Codebook

Rafael Lincoln
New York University

January 2026

Contents

1	Overview	5
2	Foundational Modules	7
2.1	<code>grids.py</code>	7
2.2	<code>quadrature.py</code>	7
2.3	<code>interpolation.py</code>	8
2.4	<code>optimize.py</code>	8
2.5	<code>markov.py</code>	8
2.6	<code>utils.py</code>	8
3	Heterogeneous Agent Tools	9
3.1	<code>backward.py</code> and <code>dp_tools.py</code>	9
3.2	<code>forward.py</code> and <code>distributions.py</code>	9
3.3	<code>steady_state.py</code>	9
3.4	<code>ssj.py</code>	9
4	Example Model	11
4.1	<code>models/aiyagari.py</code>	11

CHAPTER 1

Overview

This codebook documents the `hetmacro` package for numerical tools in macroeconomics with heterogeneity. The package is designed to be self-contained and modular, with a foundations-first architecture.

2.1 `grids.py`

- `GridSpec`: specification for one grid dimension
- `make_grid_1d`: flexible spacing and concentration
- `make_grid_nd`: build multi-dimensional grids
- `cartesian_product`, `gridmake`
- `make_asset_grid`, `double_exponential_grid`, `log_spaced_grid`

2.2 `quadrature.py`

Gaussian and Newton–Cotes quadrature rules.

- `qnwlege`, `qnwcheb`, `qnwnorm`
- `qnwlogn`, `qnwunif`, `qnwbeta`, `qnwgamma`
- `qnwsimp`, `qnwtrap`
- `qnwnorm_mv`

2.3 interpolation.py

- `interp_linear`, `interp_bilinear`
- `get_lottery`
- Spline and Chebyshev tools: `spline_coef`, `cheb_coef`, `cheb_eval`

2.4 optimize.py

Root-finding and optimization wrappers.

- `bisect`, `brentq`, `newton`, `secant`, `broyden`
- `golden_search`, `brent_min`, `nelder_mead`

2.5 markov.py

- `rouwenhorst`, `tauchen`
- `stationary_distribution`, `simulate_markov`

2.6 utils.py

- `crra_utility`, `crra_marginal`, `crra_inverse_marginal`
- `ces_aggregator`
- `compute_fixed_point`, `numerical_derivative`, `numerical_jacobian`
- `tic`, `toc`

3.1 backward.py **and** dp_tools.py

- backward_egm, backward_vfi
- policy_iteration, solve_steady_policy

3.2 forward.py **and** distributions.py

- forward_iteration, stationary_distribution
- stationary_distribution_ha, lottery_indices

3.3 steady_state.py

- household_steady_state
- market_clearing

3.4 ssj.py

- jacobian, step1_backward, J_from_F

CHAPTER 4

Example Model

4.1 `models/aiyagari.py`

- `solve_steady_state`: steady-state solution of Aiyagari model